ParaMeld：面向大语言模型训练的智能体驱动混合并行规划

Chenyang Song

上海第二工业大学计算机与信息工程学院，中国

摘要

大语言模型（LLM）智能体能够结合专业知识、工具调用与执行反馈生成优化方案，为减少分布式训练的人工调优提供了新的途径。混合并行调优需要共同选择并行度、模块分区、设备映射和运行设置；这些选择通过微批调度、通信关系及显存需求相互影响，单项参数的收益取决于其完整执行配置。本文提出 ParaMeld，一种结构与执行反馈驱动的多智能体混合并行规划方法。智能体依据模型结构和运行证据提出联合修改，约束求解将修改展开为相容的并行组合与模块布局。针对局部阶段负载难以反映整体迭代时间的问题，ParaMeld 从完整训练执行图中提取模块区间的前后向路径权重，联合求解分区边界，并通过回放结果更新后续候选的生成目标。计算迁移、通信等待和显存变化进一步用于修订下一项联合策略。在双节点 16 张 RTX 5090 上，覆盖九种稠密、MoE 及混合结构模型的原型评测汇总中，ParaMeld 相较默认配置和人工配置的平均训练吞吐分别提高 14.6% 和 5.2%。独立的四模型随机初始配置实验中，入选联合策略的实测吞吐提高 11.72%–61.25%。

关键词：LLM 智能体；分布式训练；混合并行；流水线分区；执行反馈

1 引言

LLM 智能体能够在工具交互中获取证据、提出方案并根据结果继续求解，已用于程序生成和组合优化 [1, 2]。大模型训练调优同样需要连续的分析与决策：检查模型结构和框架约束，选择并行策略，测量执行性能，再调整配置。将专业知识与这一执行过程结合，为减少人工试配、提高策略生成效率提供了机会。

混合并行决定一次训练迭代怎样使用集群资源。张量并行划分层内计算，流水线并行分配模型模块，数据并行组织输入副本，重计算和状态分片改变显存需求。这些技术具有不同的计算、通信和存储代价 [3–5]。优化任务因而需要选择它们的组合，并确定组合下的具体模块安排与运行设置。

已有自动规划系统通过结构化求解处理这些选择。分层规划将算子内部并行与阶段划分连接起来 [6]，决策树与动态规划组织多维配置空间 [7]，显存与并行联合优化进一步利用存储和执行时间之间的取舍 [8]。智能体方法则能够结合性能诊断与专业知识生成修改建议，如 PROMPTS 依据剖析结果推荐训练分片 [9]。对于完整混合并行策略，建议需要经过关联变量的展开，并利用执行结果确定后续调查方向。

这一过程首先面临配置联动。一项减少通信的修改可能增加单卡计算和存储，进而要求调整其他并行维度、重计算或模块边界。例如，降低 TP 后增加 PP 与增加 DP 具有不同的设备分组、微批数量和容量需求。模型结构也参与这些取舍：稠密模型具有首尾固定工作 [3]，MoE 的专家计算与专家存储遵循不同的规模关系 [10]，混合模型中的 Attention、Mamba 和 MLP 具有不同成本 [11]。策略生成需要表达共同修改及其保留条件，才能调查同一方向的不同实现。

另一个困难在于候选生成目标。相同模块工作量分布到不同流水线位置时，会经历不同的启动、稳态和排空依赖。最大阶段工作量因而不能充分决定完整迭代时间。在 Qwen3-14B 的 43 项布局比较中，两者在 854 对非并列比较

中的 257 对给出相反排序。仅依赖局部负载构造候选，会使生成时的优化依据与最终评价的执行目标产生差异。

我们提出 ParaMeld，将结构化的智能体规划与执行路径驱动的候选构造结合。智能体选择父策略、关联参数、模块范围及调查配额，约束工具展开相容的完整配置。对于每项并行组合，训练执行图描述全部微批的计算、通信与存储关系。系统进一步把区间前后向任务在执行路径中的出现次数转化为分区权重，联合求解开放边界，并以新布局的完整回放更新生成依据。

执行结果由此具有两项直接用途。布局搜索使用新路径修订下一项数值候选，智能体使用父子策略的计算迁移、通信等待和显存变化修订下一组联合修改。九模型训练评测考察整体系统，四模型 48 次共同工具、共同预算的搜索分别考察调查选择与布局目标，实际训练轨迹进一步展示具体策略的生成依据和执行结果。

本文的主要贡献包括：

- **结构感知的联合策略生成**。将智能体提出的优化方向表示为父策略上的关联修改，联合组织并行参数、设备映射、模块范围和运行设置，通过原生约束展开完整候选，并以执行差异指导后续生成。
- **执行路径驱动的模块布局算法**。提取完整微批调度中的前后向路径权重，构造联合边界目标，通过候选回放更新搜索中心和路径结构，使整体执行代价参与后续候选的构造。
- **多结构模型的系统实现与实验评价**。九模型原型评测汇总的平均吞吐相较默认与人工配置分别提高 14.6% 和 5.2%；受控搜索及训练案例分别检验组件作用、布局目标和具体联合策略。

2 相关工作

混合并行与自动规划。微批流水线通过在设备间分配模型工作扩展训练规模 [3, 12]，分区、阶段复制和同步调度进一步影响执行效率 [13, 14]。自动规划需要同时组织并行方式与具体执行计划。Alpa 采用算子内部和阶段之间的分层搜索 [6]；Galvatron 利用决策树和动态规划组合并行策略，其后续工作进一步处理显存负载 [7, 15]。FlexFlow 以任务模型搜索 SOAP 策略 [16]，nnScaler 通过约束表达并行变换、放置及调度 [17]。这些工作提供了组织策略空间的求解方法。ParaMeld 在完整配置上引入智能体的联合调查选择，并利用执行路径构造对应模块布局。

计算与存储取舍。训练状态分片减少本地保存的参数、梯度和优化器状态 [4, 18]，激活重建在保存开销与额外计算之间进行选择 [5, 19]。这些设置还会改变并行组合的可行范围。AdaPipe 联合优化流水线分区与重计算 [20]；Mist 通过重叠调度、符号分析和分层求解共同优化显存与并行配置 [8]。MoE Parallel Folding 为 Attention 和专家模块使用相容的不同并行映射 [21]。ParaMeld 将相关设置保留在完整候选中，根据其实际执行关系评价参数联动和模块迁移。

训练性能建模。剖析与模拟使配置能够在完整训练前进行比较。Varuna 复用测量并模拟目标配置 [22]，Proteus 和 DistSim 描述分布式任务的依赖、重叠与资源竞争 [23, 24]。计算与通信的分析成本可以分别依据 Roofline 模型和集合算法构造 [25, 26]。ParaMeld 在相容成本上回放完整微批调度，并将得到的执行路径用于候选构造，使性能评价与布局优化共同更新。

智能体与搜索控制。工具交互使语言模型能够利用外部结果连续修订决策 [1]。PROMPTS 将性能诊断与检索知识用于训练分片建议 [9]，LLM-LNS 由语言模型生成邻域选择启发式，再由求解器重构开放变量 [2]。有限评价资源的分配也是优化过程的一部分 [27]。ParaMeld 的智能体控制父策略、联合范围和调查次序，数值工具控制范围内的合法展开，完整执行结果同时影响布局生成与后续策略选择。

3 背景与问题定义

混合并行配置同时决定设备分组、模块归属和微批执行。本节先说明这些选择怎样共同影响训练，再定义 ParaMeld 求解的完整规划问题。

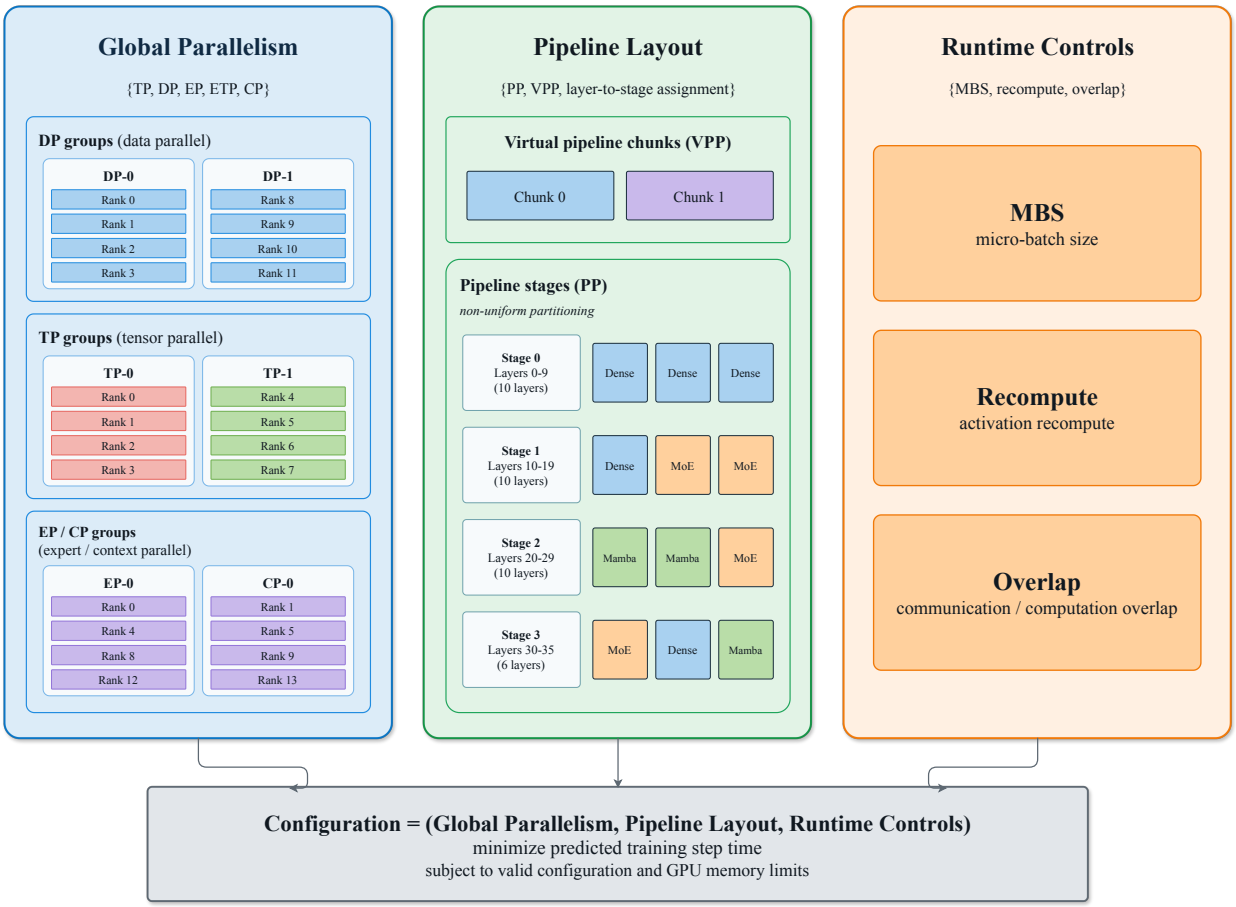


图 1　混合并行策略的决策空间。全局并行包括 TP、DP、EP、ETP 与 CP，流水线布局包括 PP、VPP 与非均匀模块分区，运行设置包括微批、重计算和通信重叠。图中的设备分组与模块数量为示意。

3.1　混合并行与模型结构

张量并行（TP）划分层内张量计算，流水线并行（PP）把连续模型区间分配到不同阶段，数据并行（DP）让多个副本处理不同样本。上下文并行（CP）划分序列，专家并行（EP）分配专家，专家张量并行（ETP）划分专家内部计算。各维度通过进程组确定计算与集合通信的参与者。

流水线还受到微批与调度的影响。全局批量在数据副本和微批之间分解，前向激活需要保存到对应反向任务使用。普通流水线每个物理阶段持有一个模型区间；虚拟流水线（VPP）使同一阶段持有多个区间，并交错执行其微批 [3]。模块迁移因而会同时改变设备工作、任务就绪与激活驻留。

模块本身也具有结构差异。稠密 Transformer 的重复层之外，嵌入、输出投影与损失为首尾阶段增加固定工作 [3, 15]。MoE 计算依赖被路由的 Token，专家存储则取决于本地持有的全部专家 [10, 28]。混合模型还组合 Attention 与状态空间计算等不同结构 [11]。本研究使用的 Nemotron-H-4B 包含 24 个 Mamba2、24 个 MLP 和 4 个 Attention 模块，分区保持它们的原始顺序。本文考察模型模块的结构差异，实验 GPU 可以同构。

图 1展示完整策略的三个关联部分：全局进程分组、流水线模块布局与运行设置。规划需要共同确定这些选择，使设备分组与具体执行安排相容。

3.2　联合决策的性能影响

Qwen3-8B 的测量提供了具体例子。图 2的 8 卡组固定 CP1、VPP1、全局批 16、微批 1、BF16 和完全重计算，并关闭 P2P 重叠。TP4/PP2/DP1 把 36 层分为两个 18 层阶段，吞吐为 9155.40 token/s，峰值显存为 19.33 GiB。

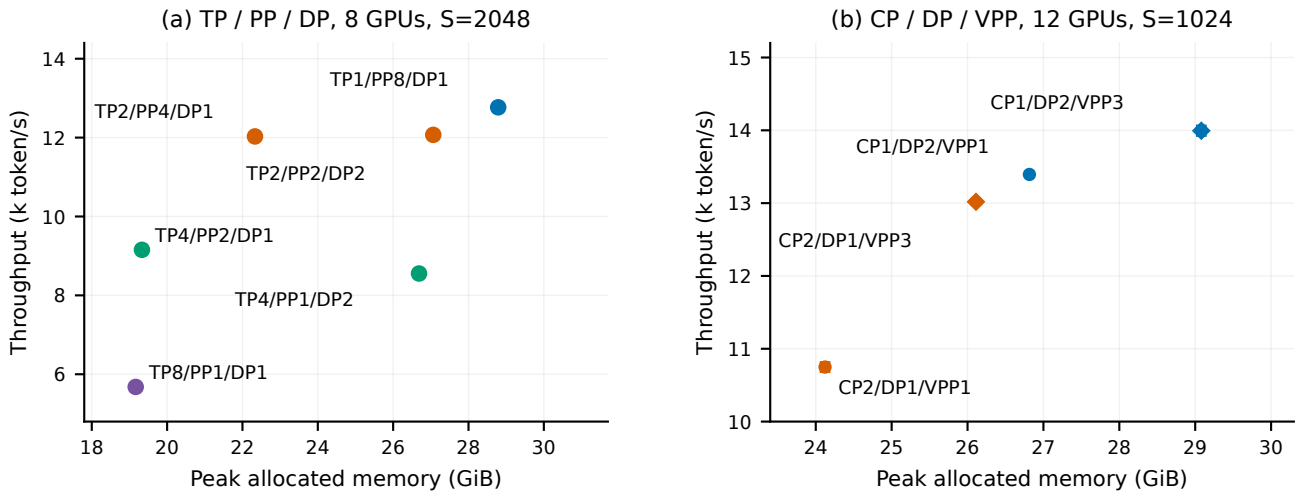


图 2　Qwen3-8B 不同混合并行配置的吞吐与显存。两组负载分别比较；右图误差线为 4 次运行的样本标准差。完整配置及测量窗口见补充表 S11。

当 TP 降至 2 时，剩余设备可以用于增加 PP 或 DP。TP2/PP4/DP1 采用四个 9 层阶段，每副本执行 16 个微批；TP2/PP2/DP2 采用两个 18 层阶段，每副本执行 8 个微批。二者吞吐分别为 12029.35 与 12069.75 token/s，显存为 22.33 与 27.06 GiB。同一 TP 修改形成相近吞吐，却具有不同的存储代价和后续调优空间。12 卡组还包含 CP、DP 与 VPP 的联动。生成器需要展开这些完整实现，才能判断一项修改的效果。

3.3　完整策略的规划目标

规划输入为模型 $\mathcal{M}$、设备拓扑 $\mathcal{D}$、固定训练负载 $\mathcal{W}$ 和搜索预算 $\mathcal{U}$。训练负载包括检查点、数据、全局批量、序列长度、精度及优化器语义。预算限制智能体调用、性能预测、GPU 测量和搜索轮次。

完整策略表示为

$$\Theta = (H, \Lambda, Z), \tag{1}$$

其中，H 包括并行维度与进程放置，Λ 包括模块区间、微批、调度、重计算及通信重叠，Z 指定执行后端和训练状态分片。仅当这些设置共同确定时，候选才对应具体的执行配置。

设 GPU 数量为 N、可分区模块数量为 D。TP、PP、稠密 DP 和 CP 的并行度分别记为 t, q, d, c，EP、ETP 和专家 DP 记为 e, t_e, d_e。每张 GPU 运行一个 Rank，进程组满足

$$tcdq = N, \qquad t_e e d_e q = N. \tag{2}$$

全局批量 B、微批大小 b、每个稠密副本的微批数量 n_μ 及 VPP 大小 v 满足

$$B = dbn_\mu, \qquad \sum_{j=0}^{qv-1} x_j = D, \quad x_j \geq 1. \tag{3}$$

$\mathbf{x}$ 按模型顺序给出各区间的模块数量，所有区间连续覆盖模型。专家副本按专家进程组处理；稠密模型使用 $e = 1$ 及 $t_e = t$。模型与后端还要求张量整除、调度支持及设备映射相容。

记满足这些约束的策略集合为 $\mathcal{F}$，实测迭代时间为 $T(\Theta)$，Rank r 的显存需求与容量分别为 $M_r(\Theta)$ 和 C_r。规划目标为

$$\min_{\Theta \in \mathcal{F}} T(\Theta), \qquad M_r(\Theta) \leq C_r \quad \forall r. \tag{4}$$

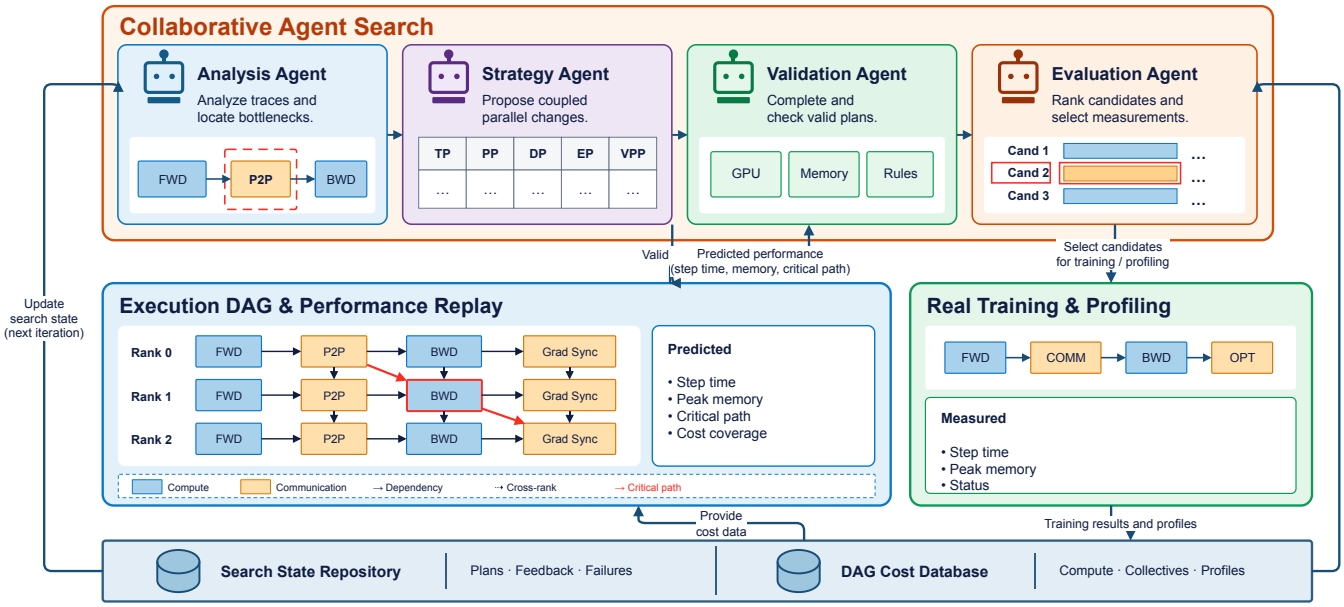


图 3　ParaMeld 的四智能体框架及执行评价与训练反馈。

ParaMeld 在预算内生成和评价候选，分别维护预测结果与成功训练策略。序列长度记为 S，预测显存采用第 4.3节的对象存活模型。

4　结构与执行反馈驱动的混合并行策略生成

混合并行策略的性能取决于工作在完整调度中的分布。对一个物理阶段累加模块成本，可以描述其总负载，却会丢失各虚拟区间的执行位置。前向任务决定后继微批何时就绪，反向任务决定前序阶段何时继续，同一设备上的不同区间还共享计算资源。因此，模块数量或阶段总工作量相近的布局，可能具有不同的迭代时间。

ParaMeld 利用这一位置差异构造候选。完整回放确定依赖与资源执行次序，各区间前后向任务在参照路径中的出现次数随后成为分区权重。逐模块成本与这些权重共同描述边界迁移对路径时间的影响，使求解器能够联合比较迁出收益与接收代价。候选改变调度后，系统重新回放并更新路径，下一项构造由新的执行关系决定。

这项布局优化嵌入智能体的完整策略搜索。智能体根据模型结构、通信和显存信息选择并行组合、模块范围及运行设置，数值生成器在选定条件内构造布局。全局条件改变时，系统重新确定微批数量、进程组与参照调度。图 3展示两项设计的联系，算法 1给出联合搜索过程。

4.1　结构感知的联合策略生成

一项优化方向通常对应多个完整实现。减少 TP 通信需要重新分配其余设备，并检查增加的单卡计算和存储；迁移输出阶段的模块需要同时选择接收位置；改变专家并行还会改变通信成员与本地专家数量。智能体据此生成包含关联变量的调查，使参数选择与具体模块安排共同进入搜索。

分析智能体将执行现象对应到模型结构。它读取模块顺序、当前完整配置、设备拓扑和任务摘要，区分重复层、首尾固定工作与结构特有模块。策略智能体在这些证据上选择父策略，提交联合参数范围、开放模块、完整布局种子及评价配额。提议同时保留目标任务和预期执行变化，供后续结果比较。

对于需要工具继续展开的调查，修改意图表示为 $I = (op, V, A, P, R)$。op 指定修改类型，V 指定目标取值，A 给出允许共同改变的变量，P 固定保留条件，R 限定取值范围。求解器保持这些语义，展开同一方向的不同合法实现。智能体也可以直接提交完整配置，并确定调查次序及至多两项完整布局种子。

修改预览将提议关联到具体执行对象。边界查询返回迁出和接收区间，维度查询返回新的进程组、计算形状与存储变化。智能体可以在提交前查询这些对象并修订范围。上下文直接保留当前策略、最好两项及最近两项结果，完整图通过任务索引按需读取。核验智能体调用原生约束工具，评价智能体组织回放和测量，四个角色共享同一策略与结果记录。

4.2 原生约束下的候选展开

联合提议的执行含义由原生约束确定。核验工具检查设备分解、张量整除、批量分解、模块覆盖与调度支持。CP-SAT 固定意图中的目标和保留条件，优先选择附加变化最少的合法补全，再排除已有组合并展开其他实现。最少附加变化使目标修改能够在接近父策略的条件下被调查，其他补全继续提供相容的联合选择。

候选身份采用后端解析后的生效设置。未开放的参数继承父策略，PP 或 VPP 变化后重新检查其运行条件。P2P 重叠、重计算和微批设置会改变相同全局并行度下的资源次序及存储，因此共同参与成本匹配。全部候选固定检查点、数据、全局批量、序列长度、精度和优化器语义；冲突返回具体条件，合法组合分别生成其对应布局。

4.3 完整策略的执行表示与评价

为了刻画区间位置的执行作用，ParaMeld 将完整策略展开为训练执行图 $G_\Theta = (V_\Theta, E_\Theta)$。计算节点对应一个微批在指定区间上的前向或反向工作，节点权重由逐模块测量或算子成本聚合。流水线传输、梯度同步与优化器更新采用显式任务表示；FSDP 配置另外包含参数收集、重新分片及 ReduceScatter。节点保留 Rank、区间、微批、形状和资源信息，使配置修改能够追踪到对应工作。

图中的依赖包括阶段之间的激活与梯度传递、原生调度规定的微批次序及参数同步条件。区间 j 映射到物理阶段 $j \bmod q$ 和虚拟编号 $\lfloor j/q \rfloor$ [3]。多个区间共享设备时，其计算资源共同计入回放，各自的前后向依赖分别保留。模型结构决定区间内部成本，Falcon-H1 等混合结构保留原生分支及汇合关系。

通信同时依赖进程组与物理放置。并行维度决定参与成员和消息大小，设备映射决定节点内或节点间路径。系统按式 (2)分别建立稠密和专家分组，并在配置变化后重新确定相应传输。模块复合测量已经包含的 TP、CP 或 EP 通信留在节点成本内，显式流水线传输和适用的梯度同步分别计入。

显存评价沿用同一执行关系。前向激活保存至对应反向任务使用，传输缓冲保存至通信及后续使用结束，临时收集参数保存至使用和重新分片完成。边界迁移改变这些对象的持有设备及存活时段，因此完整候选重新建立计算、传输和释放之间的联系。MoE 计算采用相应路由条件，持久参数存储覆盖本地持有的全部专家。

图的成本来自与实现、设备、形状、并行维度、微批、序列、精度及重计算相容的记录。直接测量优先，分析估计依据 Roofline 模型和集合通信算法构造 [25, 26]。整段 CUDA 计时按实际采集范围匹配，计算与通信按照各自覆盖范围组合；实测、估计及缺测状态分别保存。

回放依据任务就绪和资源可用性推进。单个 Rank 上的计算共享设备资源，关闭 P2P 重叠时传输也占用该资源；共享路径的并发通信共同使用带宽，计算通信并发采用适用测量提供的减速系数。完成事件释放资源并更新后继任务，最终得到迭代时间。根据回放中的依赖和资源等待 [23]，系统恢复决定各事件完成时刻的前驱关系，为路径提取提供已经确定的执行次序。

Rank r 的预测显存为

$$\widehat{M}_r = P_r + G_r + O_r + \max_{\tau} \sum_{a \in \mathcal{A}_r(\tau)} |a| + R_r. \tag{5}$$

P_r、G_r 和 O_r 分别为本地参数、梯度与优化器状态，按照模块归属和分片规则计算。$\mathcal{A}_r(\tau)$ 包含存活激活、流水线缓冲和临时收集参数的新增存储，R_r 为工作区及实测附加占用。对象按存储身份去重，持久分片和临时收集分别处理；MoE 缓冲依据适用的路由及分发数量计算。工作区或缓冲信息不足的候选保留容量未知状态。已经确认的必需

存储下界超出容量时，搜索器排除该候选；成功训练策略另外登记。

组件校准采用相容实测与估计的比值。迭代级校准在参照实测时间上增加候选与参照的模拟时间差，适用于两者残余误差相近的情形。校准条件包括模型、硬件、负载、并行维度、微批分组、精度、重计算、调度、映射和状态分片；条件变化后重新匹配测量。

4.4 执行路径驱动的候选构造

给定并行与运行条件，布局生成需要将 D 个有序模块划分为 $K = PP \cdot VPP$ 个连续区间，边界满足 $0 = b_0 < b_1 < \cdots < b_K = D$。优化这些边界时，需要知道一次模块迁移会缩短哪些执行依赖，以及接收位置会增加哪些代价。物理阶段总工作量将同一设备持有的多个区间合并，丢失了它们在微批调度中的位置。ParaMeld 保留区间身份，从完整执行路径提取这种位置差异。

考虑一条包含多个微批的执行路径。若路径经过某区间的前向任务多次，该区间减少的单次前向工作会在路径上重复累积；反向工作具有相应的累计次数。模块移入另一区间以后，新增工作则按照接收区间的次数累计。同一物理阶段上的区间可以具有不同次数，因此模块迁移的作用由其成本、迁出位置和接收位置共同决定。多条路径同时参与目标，使求解器在缩短当前较长路径时也考虑其他路径的增长。

搜索首先选择一项具有相同执行条件的完整布局作为参照。参照来自相容父布局、智能体提交的完整种子或已有策略；PP/VPP 变化时重新建立相应调度。系统在回放依赖上加入排他资源的实际执行顺序，恢复事件之间的资源等待关系。随后正向计算到达每个事件的最长前缀，反向计算从该事件出发的最长后缀，将二者连接为经过该事件的参照路径。路径中每项前后向任务均增加对应区间的累计次数。系数相同的路径保留较大的常数项，提取代价为 $O(K|V| + |E|)$。

设 $F_j(\mathbf{b})$ 和 $B_j(\mathbf{b})$ 为边界 $\mathbf{b}$ 下区间 j 的逐模块前向、反向成本之和，a^F_{pj} 与 a^B_{pj} 为路径 p 中对应任务的出现次数。生成目标为

$$\widetilde{T}(\mathbf{b}) = \max_{p \in \mathcal{P}} \left[c_p + \sum_{j=0}^{K-1} \left(a^F_{pj} F_j(\mathbf{b}) + a^B_{pj} B_j(\mathbf{b}) \right) \right]. \tag{6}$$

c_p 等于参照路径时间减去该路径的模块成本项，保留通信、固定任务及其余执行代价。在参照布局上，式 (6)复现回放的完整时间；边界变化时，它估计参照路径结构下的工作迁移。外层最大值同时约束多条路径，迁出收益和接收代价共同进入每条路径的计算。两个区间即使共享物理阶段，也保留各自的前后向系数。

为求解这一目标，生成器预先累计模型顺序上的前后向模块成本，使用相邻边界的前缀和之差表示 F_j 与 B_j。CP-SAT 共同选择开放范围内的全部边界，约束每个区间非空、模型连续覆盖及冻结模块保持归属。式 (6)作为主目标，总边界位移用于打破并列。一次求解由此可以同时迁移多个边界，比较不同区间之间相互补偿的完整布局。已评价布局加入排除集合，后续求解继续生成不同候选。

模块迁移也可能改变通信就绪、资源次序和显存峰值。这些变化通过完整回放重新计算。只有取得有效时间、容量下界未超限且完整时间改善的候选，才替换当前搜索中心；容量未知状态随结果保留。新中心重新产生路径及其权重，下一次求解随之改变。例如，原迁出区间缩短以后，接收区间可能进入新的最长路径，其前后向累计次数便参与下一次边界选择。生成目标与执行结果通过这一更新持续联系。

布局搜索在固定执行条件内使用路径代理，完整回放负责候选选择。智能体改变 TP、PP、VPP、微批或重计算以后，生成器重新匹配成本并提取参照路径，保持目标与新调度相容。该算法采用变量冻结与联合重构的求解方式 [2]，其边界目标及更新依据来自具体训练执行。采用完整接续评价部分布局的前缀引擎见补充算法 S1 [29]。

算法 1 执行反馈驱动的混合并行策略生成

输入： 模型结构、设备拓扑、固定负载、原生约束、成本与预算
输出： 最好实测策略、最好预测候选及其执行状态

核验初始策略并建立策略档案与执行记录
while 搜索预算允许 **do**
    分析智能体定位目标任务及相关执行关系
    策略智能体选择父策略，查询修改影响，提交联合候选与调查配额
    核验完整取值或补全条件意图，解析原生生效设置
    for 每项待执行的合法完整策略或调查 **do**
        评价相容参照与智能体布局，选择搜索中心
        while 本项仍有生成与评价配额 **do**
            从中心回放恢复执行路径，累计区间前后向权重
            联合求解开放边界，最小化式 (6)并排除已有布局
            匹配模块及通信成本，回放完整 DAG 并检查容量状态
            完整时间改善且容量下界未超限时更新中心
            保存候选、执行状态及已有最好结果
        end while
    end for
    比较父子执行关系，组织实际训练或需要的成本测量
    更新策略档案与剩余预算，将执行差异返回智能体
end while
return 已有最好实测记录及截止前最好预测候选

4.5 执行反馈如何改变后续策略

一次联合修改的总时间由多个变化共同形成。降低 TP 可以减少集合通信，同时增加本地模块工作；模块迁移可以缩短原区间，同时增加接收区间的工作和激活驻留。下一轮需要知道各项变化对应的执行对象。ParaMeld 按稳定模块与操作标识比较父子策略，将配置修改关联到计算形状、模块归属、通信成员和保存对象，返回相关任务、逐 Rank 计算占用、显式通信暴露及容量状态。

这些证据在两个决策层次发挥作用。固定条件内，生成器按第 4.4节更新路径目标。联合调查之间，智能体检查预期变化出现的位置，决定继承哪些设置、开放哪些关联变量，以及继续调查哪个父策略。若通信减少后接收区间的工作增加，智能体可以保留并行组合，调整接收边界；若额外通信抵消模块迁移收益，则可以共同调查设备映射与重叠设置。两类后续提议改变的配置对象不同，工具按各自完整条件重新展开和评价。

模型结构使反馈能够指向具体修改。首尾固定工作需要与可迁移层分别考虑，专家通信需要结合本地专家数量，Attention 与状态空间模块的迁移需要使用各自成本。智能体读取这些结构及任务证据，联合提交模块范围和运行设置。例如，第 5.8节的 Qwen3-14B 轨迹中，智能体查询局部反向成本后共同改变模块接收位置与选择性重计算，形成具有明确执行依据的新策略。每项提议的证据、实际生效配置和结果共同进入下一次上下文。

缺少成本时，评价智能体决定补测对象或改查其他候选。工具按操作形状、通信组和消息大小汇集缺项，并记录其影响的候选。候选能够初步排序时，工具以共享成本的高低情景检查它是否改变最好方案或 Top-K；无法排序时，提供与参照相容、候选覆盖及测量费用信息。新增测量后重新评价相关策略，CPU 搜索则返回对应测量请求。

失败按其完整执行条件进入反馈。原生冲突、容量预测、实际 OOM 和执行错误分别关联布局、微批、调度与重计算信息。后续调用通过摘要和任务索引读取相关结果，核验工具再次解析所选父策略的生效设置。

4.6 实现与预算控制

ParaMeld 基于 Megatron-LM 实现策略生成与训练执行。模型分区保留原始模块顺序，候选按照模块身份加载检查点并构造对应进程组。训练采用分布式优化器；普通流水线使用非交错 1F1B，虚拟流水线使用交错 1F1B。每项候选独立初始化优化器，测量窗口覆盖前向、反向、同步和参数更新，同时检查损失、梯度范数及更新状态。

每批 Agent 输出包含 2 至 4 项有序调查，每项可提供至多两项完整布局种子。布局求解采用整数微秒、单工作线程和至多 1.5 秒的单次求解配额，同时受剩余调查时间限制。首次求解覆盖全部授权边界，后续第 k 次构造的总位移上限为 $\min(DK, 2^k)$，$k \geq 1$。

控制器根据已完成调查的耗时与新增预测数量分配剩余配额。相同必要成本连续缺失两次时停止该项新评价；剩余时间能够容纳已有智能体调用耗时中位数及至少两次预测时，才启动下一项调用。全部模型调用、工具执行与 GPU 使用均计入费用。最终选择采用截止前完成的结果，并保留智能体提议、约束补全与布局搜索各自产生的候选。

5 实验评价

我们评价 ParaMeld 的训练吞吐、联合策略生成和执行路径优化。训练实验比较完整配置的性能与显存需求，受控搜索分别考察调查选择和布局目标，策略案例解释执行证据怎样进入智能体的后续修改。

5.1 实验设置

实验平台包含两个节点、16 张 NVIDIA RTX 5090 GPU，每张显存 32 GB。节点间通过 400 Gb/s InfiniBand 连接，节点内部使用 PCIe 5.0。主表的软件环境为 PyTorch 2.12.0a0、CUDA 13.2、NCCL 2.27 及 Megatron-LM。九模型汇总采用 BF16，随机初始配置实验采用 FP16 计算与 FP32 优化器状态。各项配对比较固定检查点、数据、精度、Adam 设置与全局批 16。

主表分别比较默认配置、人工配置与 ParaMeld。默认配置使用 Megatron 入口的参考并行组合与近似均匀分区；人工和系统从相同配置出发，均允许最多 12 次训练尝试，并可调整并行维度、VPP、微批、重计算、重叠与模块边界。历史追加搜索采用共同档案中的最好可评价策略，独立训练案例采用各自登记的共同基线。

九模型训练原型的协议记录预热 20 次、测量 50 次，每项最终配置运行 3 次，结果采用系统汇总。吞吐为 $Q = BS/T$，模型 FLOPs 利用率（MFU）按每 GPU 209.5 TFLOP/s 归一化。MoE 和混合模型按激活参数估算，MFU 用于同一模型内比较。训练原型的智能体设置记录为温度 0，每轮最多 6 次工具调用；消融使用 3 个搜索种子。各项数据来源和统计范围见第 5.9节。

5.2 训练性能

表 1与表 2分别报告 4 种稠密模型和 5 种 MoE 或混合模型。吞吐提升为 $100(Q_{\text{ParaMeld}}/Q_{\text{base}} - 1)\%$，参照对应默认或人工配置。

相对默认配置，九模型吞吐提升为 9.0%–21.7%，平均 14.6%；相对人工配置为 1.8%–10.9%，平均 5.2%。不同模型形成不同的性能与容量取舍：Qwen3-8B 提高吞吐时显存增加，MiMo-7B 同时提高吞吐并降低显存。后续案例保留每项完整策略，分别解释全局并行、调度与存储修改的作用。

表 1 稠密模型的训练性能。

模型	策略来源	MFU (%)	吞吐 (token/s)	显存 (GiB)
MiMo-7B	默认配置	24.5	17133	13.6
	人工配置	24.6	17231	14.6
	ParaMeld	**27.1**	**18932**	11.0
Qwen3-8B	默认配置	46.1	31439	15.8
	人工配置	46.6	31751	14.1
	ParaMeld	**51.7**	**35212**	23.7
Mistral-NeMo-12B	默认配置	32.3	15162	25.5
	人工配置	36.1	16912	24.8
	ParaMeld	**36.7**	**17209**	22.9
Qwen3-14B	默认配置	42.1	15886	21.5
	人工配置	47.5	17913	25.2
	ParaMeld	**48.6**	**18348**	25.2

双节点 16 卡；汇总协议为每项 3 次运行。数据来源和记录范围见第 5.9节。

表 2 MoE 与混合结构模型的训练性能。

模型	策略来源	MFU (%)	吞吐 (token/s)	显存 (GiB)
Qwen3-30B-A3B	默认配置	2.0	3377	28.2
	人工配置	2.3	3895	27.8
	ParaMeld	**2.4**	**4088**	28.3
Moonlight-16B-A3B	默认配置	8.4	17644	22.5
	人工配置	9.5	20072	24.1
	ParaMeld	**9.8**	**20643**	26.8
Nemotron-H-4B	默认配置	—	53841	21.1
	人工配置	—	55320	18.9
	ParaMeld	—	**58687**	19.3
Falcon-H1-7B	默认配置	30.7	24554	20.6
	人工配置	32.2	25736	19.1
	ParaMeld	**34.1**	**27255**	19.1
Lightning-30B-A3B	默认配置	4.3	5998	28.6
	人工配置	5.2	7156	22.9
	ParaMeld	**5.3**	**7297**	23.0

归档汇总报告双节点 16 卡、每项 3 次运行；按激活参数估算的 MFU 用于同一模型内比较。横线表示缺少一致的 FLOPs 计算记录。

表 3在两个 Qwen 模型上比较 Galvatron [7]、固定 Megatron 交错配置、人工调优与 ParaMeld。各组固定训练负载和精度，每 GPU 容量限制为 31 GiB。Galvatron 搜索 TP、PP、DP、微批与重计算，MoE 组的 EP 固定为 4。固定 Megatron 配置在 Qwen3-14B 上为 TP4/PP4/VPP2，在 Qwen3-30B-A3B 上为 TP1/PP4/CP2/EP4/VPP2；ParaMeld 同时调查相关并行维度及模块边界。

表 3　两个 Qwen 模型上的系统比较。

模型	方法	吞吐 (token/s)	显存 (GiB)	搜索 GPU 小时
Qwen3-14B	Galvatron	8901	22.6	1.42
	Megatron 1F1B	12113	27.4	–
	人工配置	17913	25.2	6.58
	ParaMeld	**18348**	25.2	2.74
Qwen3-30B-A3B	Galvatron	2249	27.1	1.96
	Megatron 1F1B	3606	28.2	–
	人工配置	3895	27.8	9.84
	ParaMeld	**4088**	28.3	4.12

记录为双节点 16 卡、3 次独立运行平均值。搜索费用包含训练、测量和失败运行；横线表示固定配置无搜索。

5.3　随机初始配置下的联合策略优化

表 4报告独立的 FP16 计算、FP32 优化器状态实验。四个模型分别从本次随机合法配置出发，智能体可以读取本次搜索产生的配置和反馈。实验固定 16 张 GPU、序列 2048 和全局批 16，每项配置预热 5 次、测量 30 次成功更新。吞吐按最慢 Rank 的平均迭代时间计算，显存采用全部 Rank 的最大已分配值；每项配置对应一次完整运行。

图 4比较四个模型调优前后的吞吐和显存。ParaMeld 生成的联合策略提高吞吐 11.72%–61.25%，对应不同的计算与存储取舍。Qwen3-8B 的变化最明显：在保持两个流水线阶段的条件下，策略将 TP 从 8 降至 4、DP 从 1 增至 2，并采用选择性重计算。其模块布局保持不变，性能差异来自完整并行与运行配置的共同调整。

Qwen3-14B 将流水线阶段由 8 个减至 4 个，同时增加 TP 并调整重计算。该策略在每阶段承担更多模块的条件下，将峰值已分配显存降低 9.70 GiB，吞吐提高 11.72%。MiMo 联合调整虚拟流水线和重计算范围，Moonlight 联合调整微批与模块归属；两者分别提高 18.46% 和 13.54%。这些结果对应全局并行、调度和存储设置的共同选择，关键配置见补充表 S16。

表 4　随机初始配置下的四模型联合策略测量。计算与模型参数采用 FP16，主参数、主梯度和 Adam 状态采用 FP32。

模型	策略	MFU(%)	吞吐 (token/s)	显存 (GiB)	吞吐提升 (%)
MiMo-7B	随机初始	19.91	13656.05	18.24	—
	ParaMeld	23.58	**16177.26**	22.82	18.46
Qwen3-8B	随机初始	14.33	10172.26	9.84	—
	ParaMeld	23.11	**16403.23**	11.94	61.25
Qwen3-14B	随机初始	25.93	10053.36	29.35	—
	ParaMeld	28.97	**11231.79**	19.65	11.72
Moonlight-16B-A3B	随机初始	7.04	14473.16	26.52	—
	ParaMeld	7.99	**16433.23**	26.76	13.54

提升相对各模型本次随机初始配置，MFU 按原始记录的模型 FLOPs 及每 GPU 209.5 TFLOP/s 计算。全部配置完成 35 次成功更新，测量窗口为其中后 30 次。

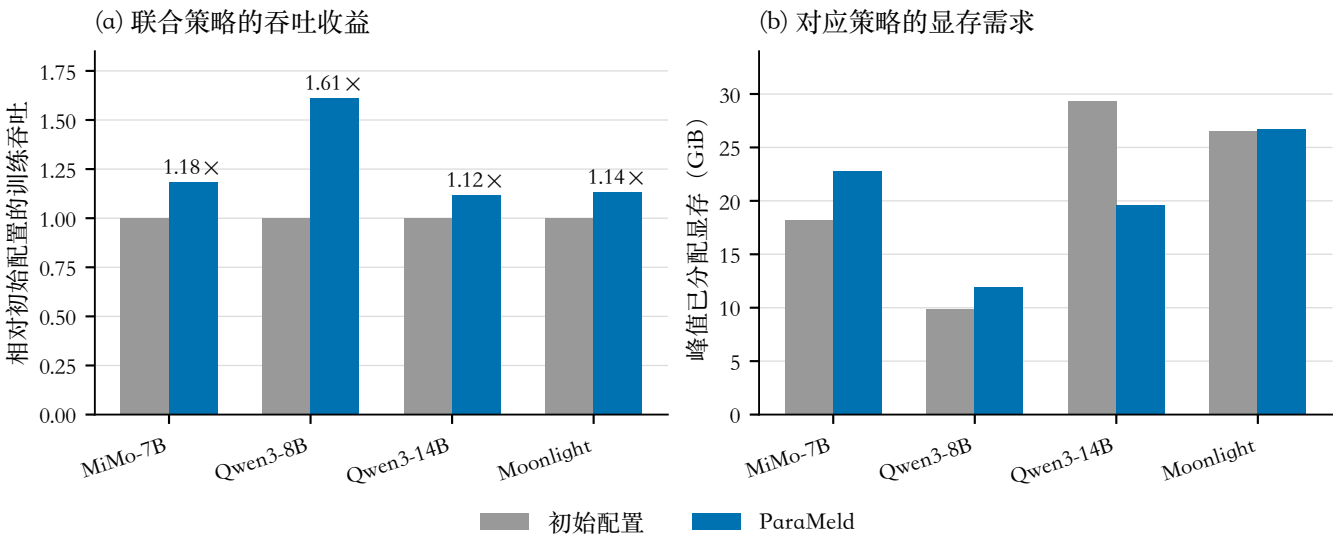


图 4　四模型联合策略的实测吞吐与显存。吞吐按各模型的随机初始配置归一化，各配置独立测量一次。

5.4　智能体组件的作用

我们使用 MiMo-7B、Nemotron-H-4B 和 Qwen3-14B 考察执行证据、独立分析和角色分工的作用。每模型每设置使用 3 个种子，每设置共 9 次搜索，最多 12 次训练尝试。目标固定为各模型默认吞吐的 1.08 倍。组件设置共享原有分区器与预测器，分别移除独立分析角色、将全部职责交给单个智能体，或移除提供给智能体的模块成本与执行依赖。

图 5给出组件比较的结果质量与条件费用。移除执行证据后，平均提升由 11.7% 降至 2.5%，9 次均未达标。相邻边界参考固定全局维度、VPP、微批和重计算，其 3 次成功均在 MiMo-7B；同权限的调查选择比较见第 5.6节。

Nemotron-H-4B 中，完整系统将末端区间从 6 个模块减少为 1 个 Mamba2 和 1 个 MLP，其余模块迁入 V5–V7，第 4 次运行达到 58687 token/s。移除执行证据时，按数量生成的布局 [6, 8, 6, 8, 6, 6, 6, 6] 达到 50222 token/s，随后提高至 54480 token/s，仍未达标。

执行证据的上下文组织另在 Moonlight 上采用三轮训练对照，具体配置、容量失败及测量费用见补充表 S15。该项实验与表 5的多模型组件比较分别统计。

表 5　智能体组件比较。结果质量按全部 9 次搜索统计；费用按达标搜索统计。

	全部搜索的结果		达标搜索的平均费用	
设置	吞吐提升/%	达标数量	训练次数	GPU 小时
完整系统	**11.7**	**9/9**	5.3	2.44
移除独立分析	6.2	4/9	8.0	3.52
合并智能体职责	4.3	2/9	9.5	4.18
移除执行证据	2.5	0/9	—	—
固定参数的启发式参考				
相邻边界移动	4.8	3/9	10.0	4.42

提升相对默认吞吐，按全部 9 次搜索统计；训练次数与 GPU 小时按达标搜索统计，各设置同时报告达标数量。完整汇总及统计来源见补充表 S1 和第 5.9节。

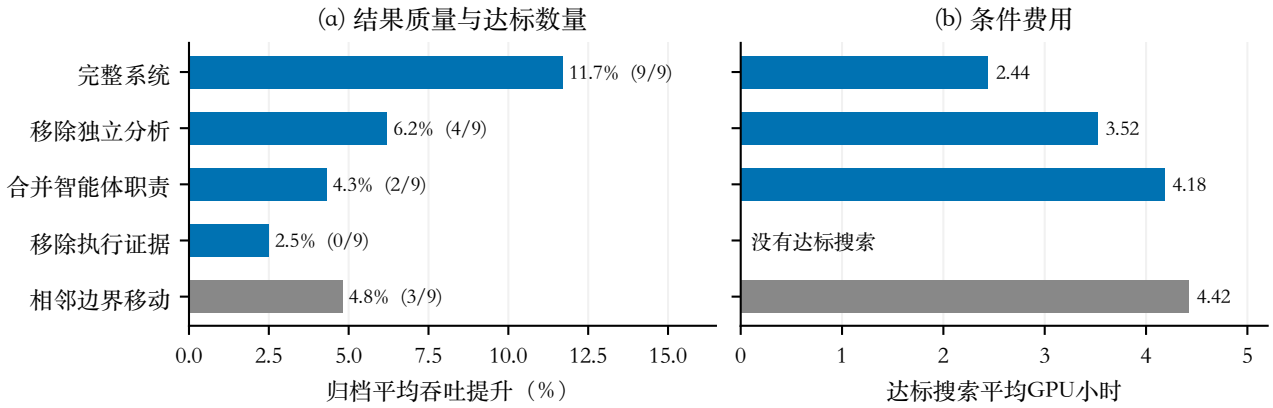


图 5　智能体组件的结果质量与条件费用。左图报告平均吞吐提升及达标数量，右图按达标搜索统计平均 GPU 小时。

5.5　执行信息与布局排序

阶段负载与完整时间的排序差异。

Qwen3-14B 的 43 项布局预测提供 854 对非并列比较，其中 257 对的最大阶段工作量排序与完整 DAG 排序相反。均匀布局与联合布局 [4, 6, 5, 5, 5, 5, 6, 4] 具有相同任务数量和 P2P 发送次数，局部工作量与完整时间仍有相反排序。补充表 S3 给出各布局的通信暴露、DAG 时间和显存模型数值；下一节固定生成工具，比较阶段工作量与执行路径两种目标。

5.6　同预算下的策略生成与搜索效率

共同工具与完整费用。

我们交叉改变调查选择器与布局目标，考察两项决策各自的作用。调查选择采用确定性覆盖或 Agent：前者按共同目录的成本可用性与覆盖组织次序并开放全部模块，后者自主提出父策略、联合参数、开放模块、完整种子与配额。布局目标采用最大物理阶段工作量或式 (6)。四组共享原生允许域、初始档案、成本、合法性工具及完整 DAG，保持边界约束、求解时间和位移规则相同。路径版本还向 Agent 提供路径系数与单模块迁移代理。

本项在 Apple M5 CPU 上从共同档案中的最好可评价策略继续搜索，评价有限预算内的追加收益。四模型、四项设置各重复三次，共 48 次搜索。每次采用 120 秒预算，最多 64 次新预测与 12 次模型调用；智能体为 `gpt-6.1-sol/low`。全部方法使用相同 PVC 组件成本和完整执行评价，调用、核验、求解及保存均计时。表 6报告截止前完成候选的最好预测时间，完整费用见补充表 S13。

表 6　历史档案追加搜索中的调查选择与布局目标。数值为 120 秒内最好预测迭代时间的三次均值，非零样本标准差用 ± 表示，单位 ms。起点为共同档案中的最好可评价策略。

	确定性覆盖选择		Agent 选择	
模型	阶段工作量	执行路径	阶段工作量	执行路径
Qwen3-14B	1488.81	1448.78	1484.27 ± 7.87	1465.83 ± 29.54
Falcon-H1-7B	1292.06	1191.78 ± 7.29	1277.85 ± 24.61	1292.06
Nemotron-H-4B	1276.85	1223.16	1244.90 ± 63.41	1266.48 ± 104.57
MiMo-7B	四组均保留参照 2537.38 ms；新并行条件存在成本缺项			

三次结果一致的单元格省略标准差。MiMo 三次运行中的四组方法均未改善参照，成本覆盖与费用见补充材料。Nemotron 使用 8 卡成本，其余使用 16 卡成本。Falcon 的阶段工作量 Agent 组有一次搜索因上下文预算耗尽终止，其终止前最好候选计入统计。

执行路径对候选生成的作用。

固定为确定性覆盖选择以后，路径目标使 Qwen、Falcon 和 Nemotron 的最好预测时间均值分别下降 2.69%、7.76% 和 4.21%。这一比较保持边界约束和求解工具相同，差别来自候选构造的目标。Qwen 中，更新后的路径权重引导首尾区间共同调整，生成物理阶段模块数量非均匀、完整预测时间更短的布局。具体边界见补充材料。

Agent 生成的联合布局与调查选择。

Nemotron 阶段工作量组的一次运行中，智能体保持 TP2/PP4/DP1/VPP2，同时减少同一物理阶段持有的两个虚拟区间的模块数量。完整评价同时计入接收区间新增工作，该候选预测时间为 1184.72 ms。确定性选择在对应运行中转向其他并行条件，体现了两种选择器对调查方向的不同分配。

三次汇总中，阶段工作量 Agent 在 Qwen、Falcon 和 Nemotron 上的均值分别低于覆盖选择 0.31%、1.10% 和 2.50%。路径组中，Nemotron 的 Agent 两次通过具体布局取得较好结果，另一次遗漏较好的全局形状，三次均值为 1266.48 ms，对应覆盖选择为 1223.16 ms。Falcon 三次生成均未覆盖确定性搜索找到的较好 PP 条件。

搜索费用与调用次数。

智能体调用占用部分搜索预算，路径组每次平均新增预测为 7.0 至 19.3 次，确定性选择为 11.3 至 25.7 次。在 Qwen、Falcon 和 Nemotron 上，阶段工作量组的智能体取得较低均值，路径组的确定性选择取得较低均值。本项 24 批有效调查均来自初始模型调用，后续操作为同次提议的工具续接。

5.7 性能模型的留出验证

留出配置为 Qwen3-14B TP2/PP8/DP1/VPP1，排除该配置的 Profile 和整步锚点。预测使用同一成本库中的 TP2/PP4/DP2 组件记录。原选择采用 VPP2 的嵌套算子计时，按采集范围匹配后采用 VPP1 的整段计时，预测由 2055.57 变为 1539.20 ms；训练记录为 1510.26 ms，绝对百分比误差由 36.11% 降至 1.92%。该独立配置的另一份运行记录对应误差 1.74%。TP8/PP2 的组件预测误差为 19.33%，PP4 返回适用通信成本需求。

5.8 联合策略的训练案例

Qwen3-14B 的一条独立 BF16 训练轨迹展示了执行证据对后续提议的作用。智能体从已完成分区优化的父策略继续，查询相邻区间的反向任务，识别到区域 6 具有较高的局部执行代价。随后它将模块 30 迁至区域 5，同时调整 MLP 重计算，形成一项关联模块位置与存储设置的联合修改。

该策略的吞吐从父策略的 12608.80 提高至 13980.51 token/s，独立重复为 14043.38 token/s。提议中的模块、接收位置及重计算设置均保存在原始响应中。补充表 S17 列出这条轨迹的布局与测量，其他完整边界及历史布局案例也集中于补充材料。

5.9 实验范围与复现材料

九模型训练与组件比较保留原系统汇总，部分逐次记录与汇总尚未完成对应。组件比较报告全部搜索的平均收益、达标数量及达标搜索的条件平均费用。配置案例按各自的模型、检查点、数据、精度、执行代码与测量窗口组织。四模型 CPU 搜索保留策略、原始响应、DAG 及费用，GPU 案例保留完整配置和逐 Rank 测量。补充材料给出多维参数扫描、同条件布局比较及 FP16 联合策略，分别对应不同的实验协议。

6 结论

ParaMeld 将智能体的联合策略选择与执行路径驱动的候选构造结合。模型结构和执行差异用于选择关联修改，完整调度的前后向权重用于求解模块边界，新布局的回放结果继续更新生成依据。九模型原型评测汇总的平均吞吐相较默认与人工配置分别提高 14.6% 和 5.2%；受控搜索及训练案例给出了布局目标、联合策略和执行性能之间的具体联系。

参考文献

[1] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing Reasoning and Acting in Language Models. In: *The Eleventh International Conference on Learning Representations*. 2023.

[2] Ye H, Xu H, Yan A, Cheng Y. Large Language Model-driven Large Neighborhood Search for Large-Scale MILP Problems. In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. PMLR. 2025:72131–72180.

[3] Narayanan D, Shoeybi M, Casper J, et al. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 2021:1–15

[4] Rajbhandari S, Rasley J, Ruwase O, He Y. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. In: *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. 2020:1–16

[5] Korthikanti VA, Casper J, Lym S, et al. Reducing Activation Recomputation in Large Transformer Models. In: *Proceedings of Machine Learning and Systems*. Vol. 5. 2023:341–353.

[6] Zheng L, Li Z, Zhang H, et al. Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning. In: *16th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association. 2022:559–578.

[7] Miao X, Wang Y, Jiang Y, et al. Galvatron: Efficient Transformer Training over Multiple GPUs Using Automatic Parallelism. *Proceedings of the VLDB Endowment*. 2022;16(3):470–479. doi: 10.14778/3570690.3570697

[8] Zhu Z, Giannoula C, Andoorveedu M, et al. Mist: Efficient Distributed Training of Large Language Models via Memory-Parallelism Co-Optimization. In: *Proceedings of the Twentieth European Conference on Computer Systems*. 2025:1298–1316

[9] Ding Y, Han R, Zhang X, Chen X. PROMPTS: PeRformance Optimization via Multi-Agent Planning for LLM Training and Serving. *Proceedings of Machine Learning and Systems*. 2026;8:1010–1039.

[10] Rajbhandari S, Li C, Yao Z, et al. DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale. In: *Proceedings of the 39th International Conference on Machine Learning*. Vol. 162. PMLR. 2022:18332–18346.

[11] Dao T, Gu A. Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality. In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. PMLR. 2024:10041–10071.

[12] Huang Y, Cheng Y, Bapna A, et al. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. In: *Advances in Neural Information Processing Systems*. Vol. 32. Curran Associates, Inc. 2019.

[13] Narayanan D, Harlap A, Phanishayee A, et al. PipeDream: Generalized Pipeline Parallelism for DNN Training. In: *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. 2019:1–15

[14] Fan S, Rong Y, Meng C, et al. DAPPLE: A Pipelined Data Parallel Approach for Training Large Models. In: *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. 2021:431–445

[15] Wang Y, Jiang Y, Miao X, et al. Improving Automatic Parallel Training via Balanced Memory Workload Optimization. *IEEE Transactions on Knowledge and Data Engineering*. 2024;36(8):3906–3920. doi: 10.1109/TKDE.2024.3370614

[16] Jia Z, Zaharia M, Aiken A. Beyond Data and Model Parallelism for Deep Neural Networks. In: *Proceedings of Machine Learning and Systems*. Vol. 1. 2019:1–13.

[17] Lin Z, Miao Y, Zhang Q, et al. nnScaler: Constraint-Guided Parallelization Plan Generation for Deep Learning Training. In: *18th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association. 2024:347–363.

[18] Zhao Y, Gu A, Varma R, et al. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. *Proceedings of the VLDB Endowment*.

2023;16(12):3848–3860. doi: 10.14778/3611540.3611569

[19] Jain P, Jain A, Nrusimha A, et al. Checkmate: Breaking the Memory Wall with Optimal Tensor Rematerialization. In: *Proceedings of Machine Learning and Systems*. Vol. 2. 2020:497–511.

[20] Sun Z, Cao H, Wang Y, et al. AdaPipe: Optimizing Pipeline Parallelism with Adaptive Recomputation and Partitioning. In: *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 2024:86–100

[21] Liu D, Yan Z, Yao X, et al. MoE Parallel Folding: Heterogeneous Parallelism Mappings for Efficient Large-Scale MoE Model Training with Megatron Core. *arXiv preprint arXiv:2504.14960*. 2025.

[22] Athlur S, Saran N, Sivathanu M, Ramjee R, Kwatra N. Varuna: Scalable, Low-cost Training of Massive Deep Learning Models. In: *Proceedings of the Seventeenth European Conference on Computer Systems*. Association for Computing Machinery. 2022:472–487

[23] Duan J, Li X, Xu P, et al. Proteus: Simulating the Performance of Distributed DNN Training. *IEEE Transactions on Parallel and Distributed Systems*. 2024;35(10):1867–1878. doi: 10.1109/TPDS.2024.3443255

[24] Lu G, Chen R, Wang Y, et al. DistSim: A Performance Model of Large-Scale Hybrid Distributed DNN Training. In: *Proceedings of the 20th ACM International Conference on Computing Frontiers*. 2023:112–122

[25] Williams S, Waterman A, Patterson D. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Communications of the ACM*. 2009;52(4):65–76. doi: 10.1145/1498765.1498785

[26] Thakur R, Rabenseifner R, Gropp W. Optimization of Collective Communication Operations in MPICH. *The International Journal of High Performance Computing Applications*. 2005;19(1):49–66. doi: 10.1177/1094342005051521

[27] Li L, Jamieson K, DeSalvo G, Rostamizadeh A, Talwalkar A. Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization. *Journal of Machine Learning Research*. 2018;18(185):1–52.

[28] Zhai M, He J, Ma Z, Zong Z, Zhang R, Zhai J. SmartMoE: Efficiently Training Sparsely-Activated Models through Combining Offline and Online Parallelization. In: *2023 USENIX Annual Technical Conference*. 2023:961–975.

[29] Bertsekas DP, Tsitsiklis JN, Wu C. Rollout Algorithms for Combinatorial Optimization. *Journal of Heuristics*. 1997;3(3):245–262.